

BICEC VeriPass — Vue d'ensemble

Stage Xp-X5

- [BICEC VeriPass — Vue d'Ensemble](#)
 - [Table des matières](#)
 - [0. Couverture et cadrage](#)
 - [0.1 Comment lire ce document](#)
 - [0.2 Résumé exécutif \(one-pager\)](#)
 - [0.3 Périmètre explicite et hors-périmètre verrouillé](#)
 - [0.4 Légende visuelle des diagrammes](#)
 - [1. Niveau 1 — System Context \(C4 L1\)](#)
 - [1.1 Diagramme de contexte système](#)
 - [1.2 Zones de confiance et frontières de sécurité](#)
 - [1.3 Texte — Positionnement produit et invariants métier](#)
 - [2. Niveau 2 — Containers \(C4 L2\)](#)
 - [2.1 Vue d'ensemble des containers](#)
 - [2.2 Topologie des volumes Docker externes](#)
 - [2.3 Routage Nginx public](#)
 - [2.4 Tableau détaillé des 13 containers](#)
 - [3. Niveau 3 — Components \(C4 L3\)](#)
 - [3.1 Backend — entry point et router API](#)
 - [3.2 Backend — modules métier](#)
 - [3.3 Backend — core/*, db/*, services/*, tasks/*](#)
 - [3.4 Mobile PWA — route map, contexts, services](#)
 - [3.5 Backoffice SPA — route map, pages, services](#)
 - [4. Niveau 4 — Code \(C4 L4\) — parcours critiques](#)
 - [4.1 Machine d'état KYC et access_level](#)
 - [4.2 Séquence du parcours KYC complet](#)
 - [4.3 Pipeline OCR et biométrie](#)
 - [4.4 Topologie Celery et beat schedule](#)
 - [4.5 Modèle de données consolidé](#)
 - [4.6 Séquence de la revue backoffice](#)
 - [4.7 Cycle de vie d'un dossier \(vue gitGraph\)](#)
 - [4.8 Matrice RBAC](#)
 - [4.9 Happy-path end-to-end](#)
 - [4.10 Handoff mobile aval \(app-link\)](#)
 - [5. Opérations et handover](#)
 - [5.1 Topologie de déploiement](#)
 - [5.2 Topologie backup et rétention](#)
 - [5.3 Carte d'observabilité](#)
 - [5.4 Où changer quoi](#)
 - [5.5 Invariants critiques](#)
 - [5.6 Commandes smoke \(copier-coller\)](#)

- [5.7 Zones à risque connu](#)
- [5.8 Glossaire FR/EN](#)
- [5.9 Versioning et changelog](#)
- [6. Annexes](#)
 - [Annexe A — Index croisé master ↔ docs détaillées](#)
 - [Annexe B — Rendu SVG et PDF — comment régénérer](#)
 - [Annexe C — Note d'autorité documentaire](#)

BICEC VeriPass — Vue d'Ensemble

Architecture consolidée C4 (Context → Container → Component → Code)



Logo BICEC

Champ	Valeur
Version du document	1.0.0
Date de génération	2026-06-02
Langue principale	Français
Statut	Source of truth — handover
Audience	Équipe IT BICEC, mainteneurs, successeurs, agents de revue
Périmètre applicatif	KYC onboarding, stockage d'identité vérifiée, garde-fou d'éligibilité pour les apps mobiles aval
Hors-périmètre (verrouillé)	DGI, Sopra Amplitude, core banking — voir § 0.3

Table des matières

- [0. Couverture et cadrage](#)
 - 0.1 [Comment lire ce document](#)
 - 0.2 [Résumé exécutif \(one-pager\)](#)
 - 0.3 [Périmètre explicite et hors-périmètre verrouillé](#)

- 0.4 [Légende visuelle des diagrammes](#)
 - [1. Niveau 1 — System Context \(C4 L1\)](#)
 - 1.1 [Diagramme de contexte système](#)
 - 1.2 [Zones de confiance et frontières de sécurité](#)
 - 1.3 [Texte — Positionnement produit et invariants métier](#)
 - [2. Niveau 2 — Containers \(C4 L2\)](#)
 - 2.1 [Vue d'ensemble des containers](#)
 - 2.2 [Topologie des volumes Docker externes](#)
 - 2.3 [Routage Nginx public](#)
 - 2.4 [Tableau détaillé des 13 containers](#)
 - [3. Niveau 3 — Components \(C4 L3\)](#)
 - 3.1 [Backend — entry point et router API](#)
 - 3.2 [Backend — modules métier](#)
 - 3.3 [Backend — core/*, db/*, services/*, tasks/*](#)
 - 3.4 [Mobile PWA — route map, contexts, services](#)
 - 3.5 [Backoffice SPA — route map, pages, services](#)
 - [4. Niveau 4 — Code \(C4 L4\) — parcours critiques](#)
 - 4.1 [Machine d'état KYC et access_level](#)
 - 4.2 [Séquence du parcours KYC complet](#)
 - 4.3 [Pipeline OCR et biométrie](#)
 - 4.4 [Topologie Celery et beat schedule](#)
 - 4.5 [Modèle de données consolidé](#)
 - 4.6 [Séquence de la revue backoffice](#)
 - 4.7 [Cycle de vie d'un dossier \(vue gitGraph\)](#)
 - 4.8 [Matrice RBAC](#)
 - 4.9 [Happy-path end-to-end](#)
 - 4.10 [Handoff mobile aval \(app-link\)](#)
 - [5. Opérations et handover](#)
 - 5.1 [Topologie de déploiement](#)
 - 5.2 [Topologie backup et rétention](#)
 - 5.3 [Carte d'observabilité](#)
 - 5.4 [Où changer quoi](#)
 - 5.5 [Invariants critiques](#)
 - 5.6 [Commandes smoke \(copier-coller\)](#)
 - 5.7 [Zones à risque connu](#)
 - 5.8 [Glossaire FR/EN](#)
 - 5.9 [Versioning et changelog](#)
 - [6. Annexes](#)
 - A [Index croisé master ↔ docs détaillées](#)
 - B [Rendu SVG et PDF — comment régénérer](#)
 - C [Note d'autorité documentaire](#)
-

0. Couverture et cadrage

0.1 Comment lire ce document

Ce document est conçu pour être lu **de haut en bas**. À chaque section, vous trouverez :

1. Un **diagramme Mermaid** (rendu natif dans GitHub, VS Code, Obsidian, et la plupart des viewers Markdown). Une copie SVG est disponible sous [docs/c4-architecture/diagrams/](#) pour impression et inclusion dans un PDF.
2. Un **texte explicatif** court (rôle, acteurs, frontières).
3. Un **cross-link** vers la documentation détaillée correspondante dans docs/ (la « doc de référence »). Cette doc de référence reste la vérité de détail ; le présent document est la vérité d'ensemble.

Si vous n'avez que 10 minutes, lisez § 0.2, § 1.1, § 2.1, § 3.1, § 4.1, § 4.9.

Si vous avez 1 heure, lisez toutes les sections C4 dans l'ordre (sections 1 → 2 → 3 → 4), puis § 5.4 et § 5.5.

Si vous prenez en main le projet, lisez tout, puis exécutez les commandes de § 5.6.

0.2 Résumé exécutif (one-pager)

BICEC VeriPass est une **plateforme souveraine de KYC digital** pour la BICEC. Elle remplace un parcours d'onboarding en agence par un parcours mobile-first où un client capture ses pièces d'identité, confirme l'extraction OCR, passe une épreuve de vivacité, soumet son consentement, et attend une **décision humaine** par un agent backoffice.

Trois surfaces utilisateurs, un cœur backend :

Surface	URL	Utilisateur	Rôle
PWA Mobile	https://localhost/mobile/	Marie (cliente)	Capture KYC, dashboard, handoff vers apps aval
SPA Backoffice	https://localhost/back-office/	Jean / Thomas / Sylvie / Admin IT	Validation KYC, conformité AML, centre de commande, administration
API + docs	https://localhost/api/v1/docs	Développeurs / Intégrateurs	FastAPI async, OpenAPI

Règle métier fondamentale : l'automatisation prépare le dossier, mais l'approbation KYC est **décidée par un humain**. VeriPass stocke l'identité KYC vérifiée dans sa propre base et agit comme un **garde-fou d'authentification et d'éligibilité** avant d'envoyer l'utilisateur vers les autres applications mobiles BICEC (BI PAY, BICEC Mobile-Banking, BICEC Wallet).

Stack technique : Python 3.11 + FastAPI + SQLAlchemy async + Alembic + PostgreSQL 17 + Redis 7 + Celery + React 19 + TypeScript + Vite + TanStack Query + Docker Compose + Nginx + PaddleOCR + GLM-OCR + MediaPipe + MiniFASNet + DeepFace + Sentry + Flower.

Volumes Docker externes critiques (NE PAS supprimer) : code_db_storage, code_documents_storage, code_db_backups, code_models_storage.

0.3 Périmètre explicite et hors-périmètre verrouillé

Note d'autorité. Aucun document historique, prototype, ou note d'agent ne peut redéfinir ce périmètre. Voir l'[Annexe C](#).

Dans le périmètre (in-scope) :

- Onboarding KYC mobile (CNI, selfie/liveness, justificatif de domicile, NIU, consentements, signature).
- Stockage d'identité vérifiée en PostgreSQL + fichiers KYC en volume externe.
- Backoffice de validation, AML/CFT, gestion des agences, administration des agents.
- Authentification client (OTP, PIN, passkey/WebAuthn) et agent (email/mot de passe).
- Garde-fou d'éligibilité pour les apps mobiles aval via app-link / fallback App Store / Google Play.
- Mode souverain / offline-first (modèles IA locaux, pas de dépendance réseau).
- Backups PostgreSQL quotidiens + archives chiffrées des documents KYC.

Hors-périmètre, verrouillé (NON) :

- × **Intégration DGI** (Direction Générale des Impôts). Aucune couche DGI, aucun mock DGI, aucune route /dgi/*.
- × **Intégration Sopra Amplitude**. Aucun client Amplitude, aucun état de provisioning Amplitude, aucune route /amplitude/* ou équivalent.
- × **Intégration core banking**. Pas d'adaptateur core banking. Les flux monétaires (cartes, virements, épargne) sont des **coquilles produit locales** (modules/banking/) et ne représentent pas un adaptateur réel.
- × **Provisioning de compte aval dans une banque tierce**. L'accès aux apps aval (BI PAY, Mobile-Banking, Wallet) se fait par **handoff mobile OS-aware** (app-link → fallback store), pas par provisioning côté VeriPass.

0.4 Légende visuelle des diagrammes

Tous les diagrammes de ce document respectent les conventions suivantes :

Élément	Signification
Gris	Niveau 1 — System Context (personnes, systèmes externes)
Bleu	Niveau 2 — Containers (processus / conteneurs Docker)
Orange	Niveau 3 — Components (modules / fichiers / services)
Vert	Niveau 4 — Code (parcours critiques, états, séquences)
Flèche pleine	Appel synchrone
Flèche pointillée	Appel asynchrone ou événement
Double flèche	Flux de données persistant
subgraph . . . end	Limite de couche ou de domaine

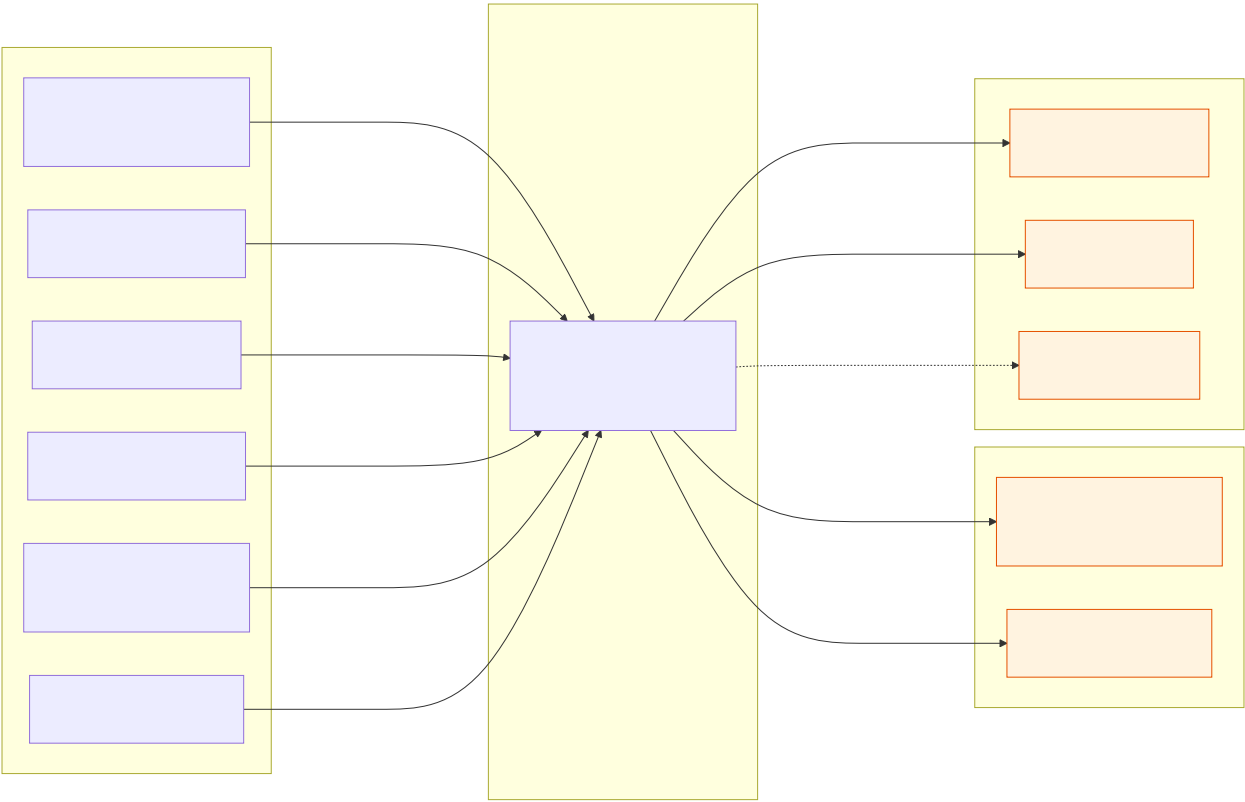
Chaque diagramme est numéroté (X.Y) et un cross-link pointe vers la doc détaillée correspondante.

1. Niveau 1 — System Context (C4 L1)

Le diagramme de contexte montre **qui** utilise VeriPass, **avec quoi** il interagit, et **où** se trouvent les frontières de confiance.

1.1 Diagramme de contexte système

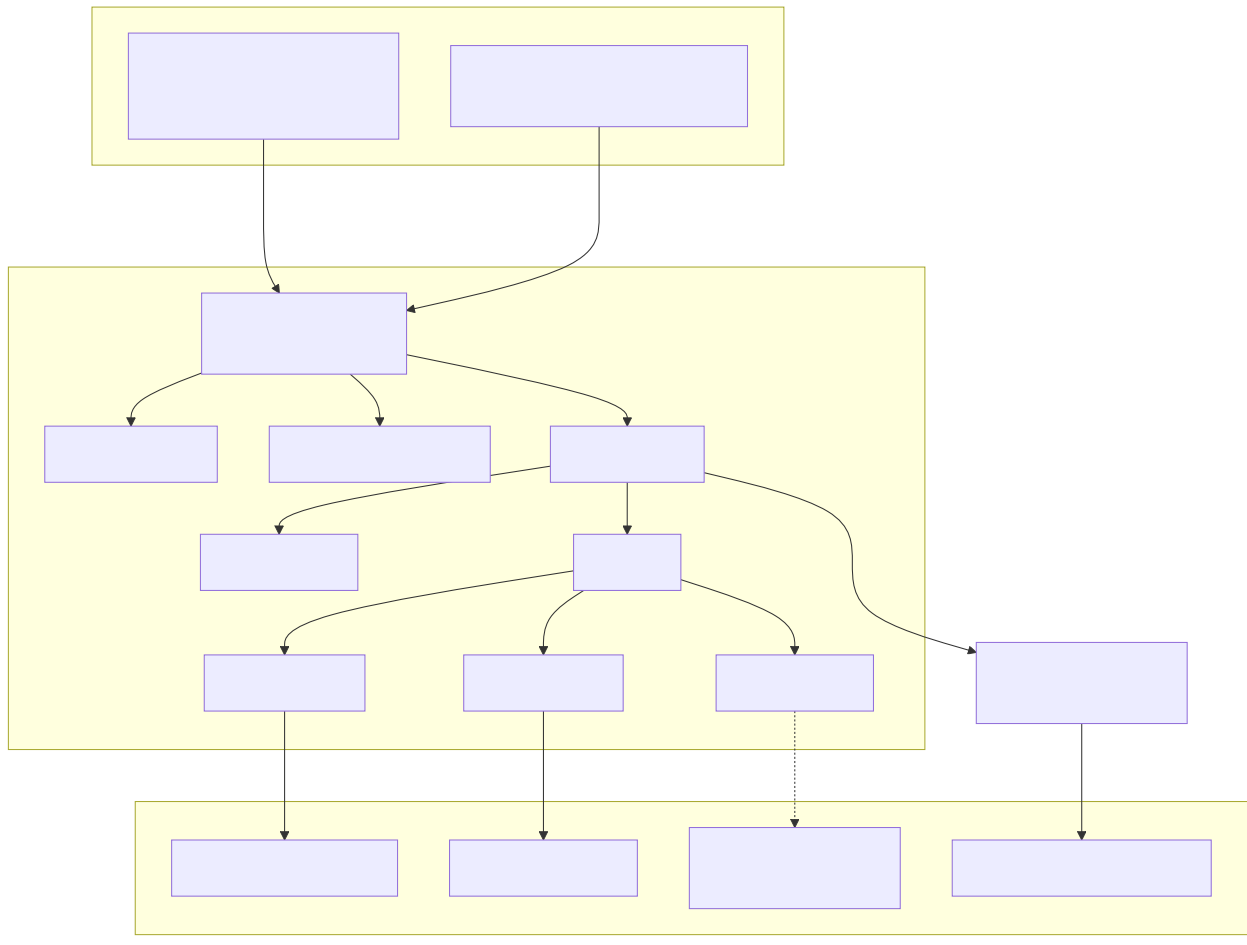
Figure 1 — c4-0-1-table



Lecture du diagramme. Marie utilise VeriPass depuis son navigateur mobile pour faire son KYC. Jean, Thomas, Sylvie, et Admin IT utilisent VeriPass depuis leurs navigateurs backoffice. Quand un dossier est approuvé, VeriPass agit comme un **garde-fou** : il ne fait pas d'intégration avec les apps aval, il indique au mobile quelle app-link essayer et quel store fallback utiliser. Côté externe, VeriPass délègue uniquement l'envoi d'OTP/SMS, la synchronisation PEP/sanctions, et (optionnellement) le fallback OCR cloud. Tout le reste tourne en local.

1.2 Zones de confiance et frontières de sécurité

Figure 2 — c4-1-2-zones



Lecture du diagramme. La seule frontière de confiance entre l'utilisateur et VeriPass est **Nginx en HTTPS**. Tout le reste (API, DB, Redis, workers) est interne au réseau Docker veripass-net. Les appels aux systèmes externes sont sortants et limités à des usages précis (OTP, sanctions, OCR cloud optionnel). Les apps aval sont atteintes par **redirection côté mobile**, pas par intégration côté VeriPass.

1.3 Texte — Positionnement produit et invariants métier

Positionnement. VeriPass est **un système d'onboarding KYC et de stockage d'identité vérifiée**, pas un adaptateur vers les systèmes internes de la banque. Il ne crée pas de compte dans un core banking ; il décide qu'un utilisateur peut, après approbation humaine, accéder aux apps mobiles aval via des liens OS-aware.

Invariants métier (à ne jamais transgresser) :

1. L'approbation KYC est **toujours humaine**. Aucune transition vers APPROVED ne peut être déclenchée par un processus automatique.
2. Le status (cycle de vie) et l'`access_level` (permission) sont **deux dimensions orthogonales**. Ne jamais les fusionner.
3. L'accès aux apps aval est un **handoff mobile**, pas une intégration backend.
4. Le status APPROVED mappe (par défaut) sur LIMITED_ACCESS. L'accès FULL_ACCESS est un cas séparé qui n'est pas dans le périmètre actuel.
5. Le status FRAUD_SUSPECT mappe (par défaut) sur DISABLED.
6. PENDING_INFO reste **éditable** côté client pour permettre la resoumission des pièces demandées.

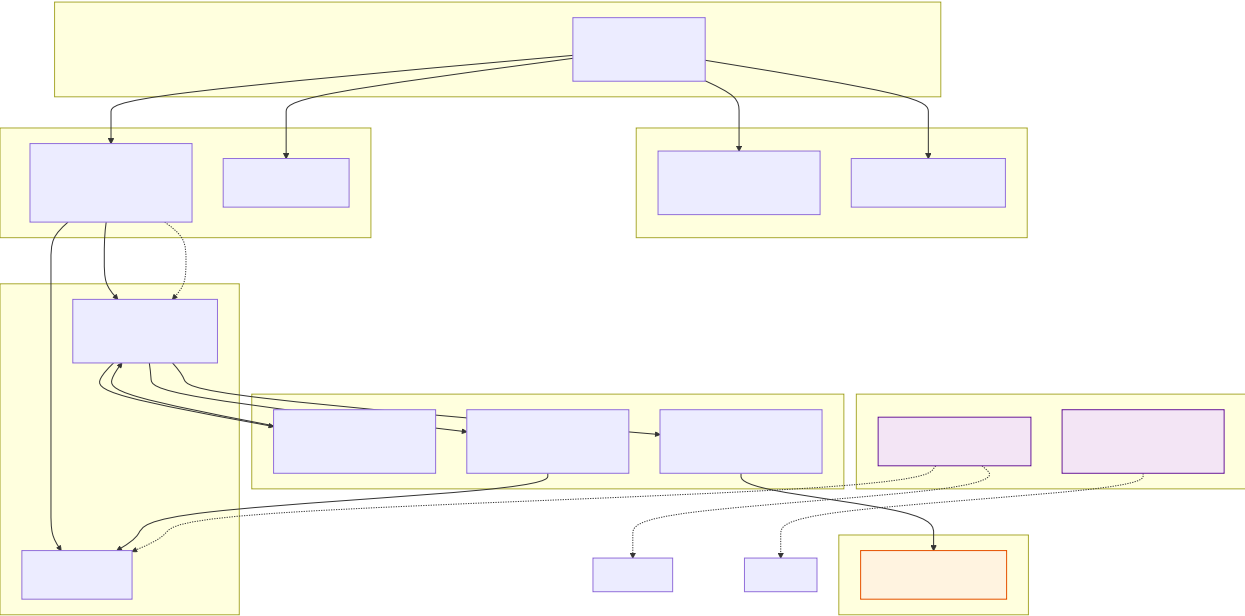
Cross-link. Détail dans [docs/project-overview.md](#) § « State And Access Model » et [docs/data-models.md](#) § « KYC Tables ». Diagramme équivalent dans [docs/diagrams/statechart-access.md](#).

2. Niveau 2 — Containers (C4 L2)

Le diagramme de containers montre **les processus conteneurisés** et leurs relations.

2.1 Vue d'ensemble des containers

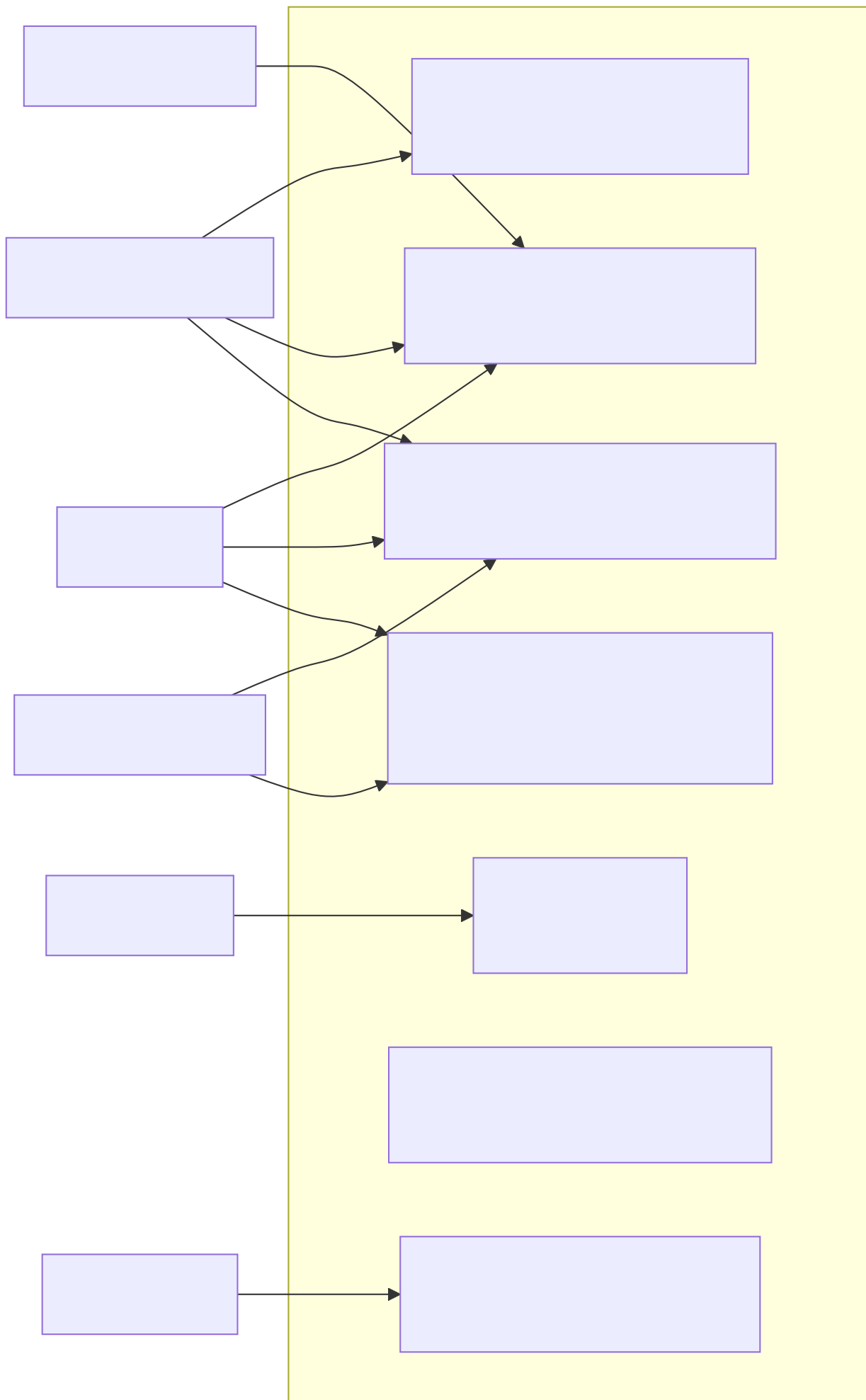
Figure 3 — c4-1-3-texte



Lecture du diagramme. 13 conteneurs au total (hors init), organisés en 5 zones : Edge (Nginx), Static (PWA + Backoffice), App (API + Flower), Data (Postgres + Redis), Workers (3 Celery), Dev (Mailpit). Le conteneur `vp_storage_init` est un one-shot qui fixe les permissions des volumes au démarrage. `vp_model_init` n'est utilisé que derrière le profil `online-init` pour bootstrapper les modèles (en mode souverain, les modèles sont déjà montés depuis l'hôte via `OFFLINE_MODELS_DIR`).

2.2 Topologie des volumes Docker externes

Figure 4 — c4-2-2-topologie

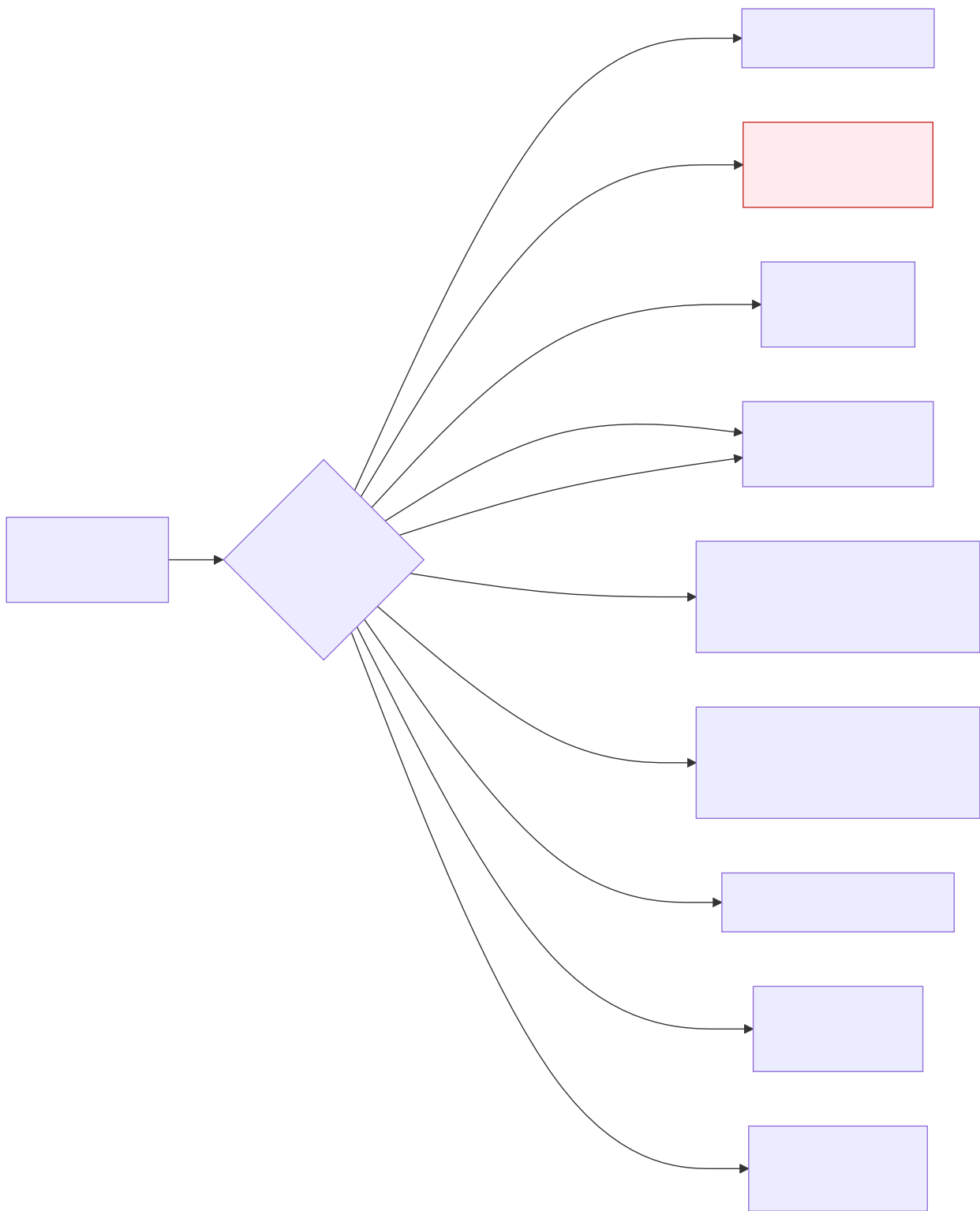


Lecture du diagramme. Trois volumes sont **persistants et critiques** :
code_db_storage (données métier), code_documents_storage (fichiers KYC),
code_db_backups (sauvegardes). Tout script de cleanup (docker prune, docker
compose down -v) qui toucherait à ces volumes est **interdit sauf demande
explicite**.

Cross-link. Détail dans [docs/deployment-guide.md](#) § « External
Volumes » et [docs/operations-runbook.md](#) § « Disk Space Is Low ».

2.3 Routage Nginx public

Figure 5 — c4-2-3-routage



Lecture du diagramme. Nginx est la seule porte d'entrée publique. Il applique :

- **Redirection HTTP → HTTPS** systématique.
- **Rate-limit strict** sur /api/v1/auth/* (zone dédiée).
- **Rate-limit normal** sur le reste de l'API.
- **Headers de sécurité** centralisés (security_headers.conf).
- **Fallback SPA** : toute URL non trouvée sous /mobile/* ou /back-office/* renvoie index.html pour gérer les deep-links (ex : /back-office/validation/dossier/abc-123).
- **Timeouts étendus** sur les routes OCR et liveness.
- **Redirection /backoffice/ → /back-office/** (alias historique).
- **Proxy Sentry** : /api/v1/sentry-proxy/* et /sentry-proxy/* permettent de garder les envois Sentry en same-origin.

2.4 Tableau détaillé des 13 containers

Service	Conteneur	Image de base	Ports hôte	Volumes montés
nginx	vp_nginx	nginx-unprivileged (build local)	443:8443, 80:8080, 127.0.0.1:8081:8081	nginx.conf:ro, security_headers.conf:ro, ssl/:ro
pwa	vp_pwa	nginx-unprivileged	(interne)	(statique unique)
backoffice	vp_backoffice	nginx-unprivileged	(interne)	(statique unique)
api	vp_api	python:3.11-slim + backend	8001:8000 (debug)	code_db_storage, code_documents, code_models_storage, code_models_public
postgres	vp_postgres	postgres:17	15432:5432 (debug)	code_db_storage
redis	vp_redis	redis:7	16379:6379 (debug)	redis_data
celery_ocr	vp_celery_ocr	backend (même image que api)	(interne)	code_documents, code_models_storage
celery_notifications	vp_celery_notif	backend	(interne)	code_db_storage
celery_beat	vp_celery_beat	backend	(interne)	code_db_storage, code_db_backup

Service	Conteneur	Image de base	Ports hôte	Volumes mon
mailpit	vp_mailpit	mailpit	8025:8025 (UI), 1025:1025 (SMTP)	code_document var/run/docke
flower	vp_flower	backend (même image)	(interne)	—
storage_init	vp_storage_init	alpine	—	code_db_stora code_document code_models_s code_db_back
model_init	vp_model_init	python:3.11- slim	—	code_models_s

⚠ **Docker socket sur celery_beat** : nécessaire pour les tâches de prune disk ; à restreindre en production (voir § 5.7).

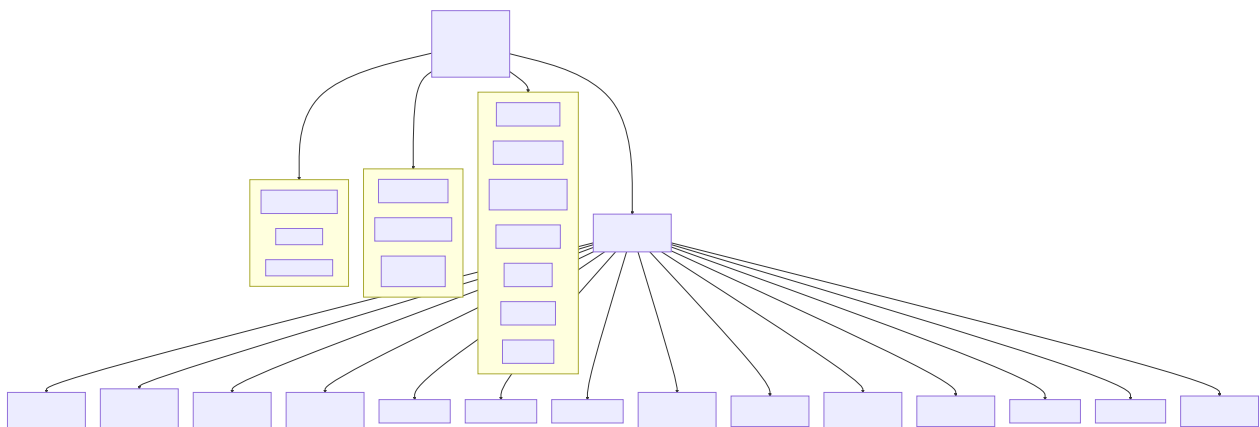
Cross-link. Détail complet dans [docs/deployment-guide.md](#) et code/`docker-compose.yml`.

3. Niveau 3 — Components (C4 L3)

Le diagramme de composants montre les **modules applicatifs** et leurs relations.

3.1 Backend — entry point et router API

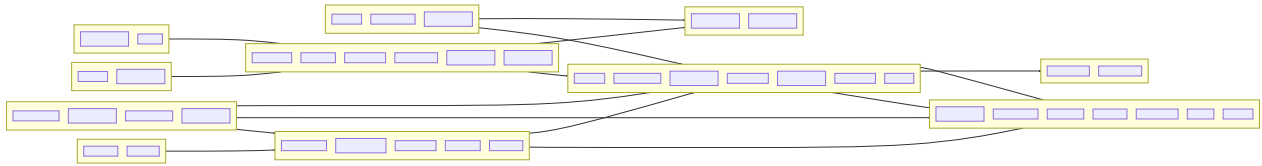
Figure 6 — c4-2-4-tableau



Lecture du diagramme. L'application est un **monolithe modulaire FastAPI** organisé en domaines (modules/*). Chaque module suit la convention `router.py + service.py + models.py + schemas.py`. Le routeur racine (`api/v1/router.py`) inclut tous les routeurs de modules. Les couches transverses (`core/`, `db/`, `services/`, `tasks/`) sont partagées par les modules.

3.2 Backend — modules métier

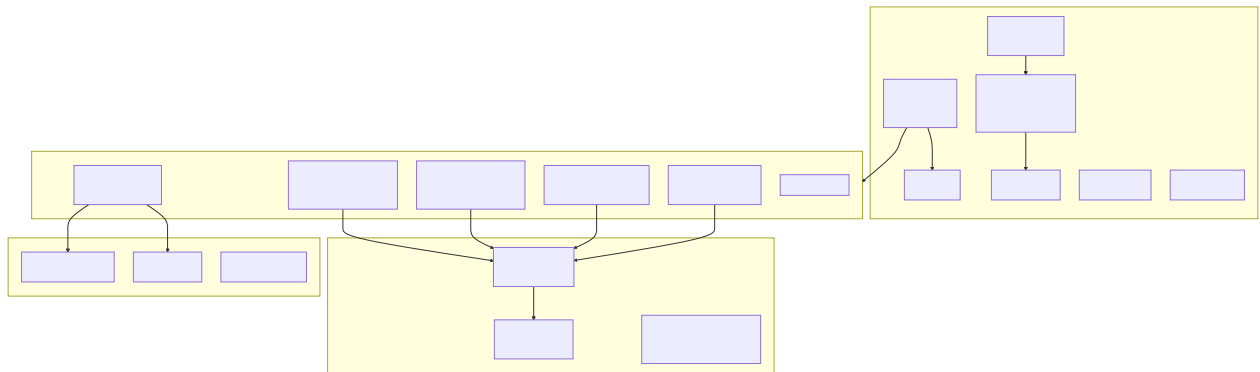
Figure 7 — c4-3-2-backend



Lecture du diagramme. Les modules sont fortement interconnectés : kyc est le module central (tous les autres le lisent ou l'écrivent), auth est traversé par toutes les routes protégées, backoffice orchestre la validation humaine. banking est volontairement isolé des modules auth/kyc au-delà de la lecture de access_level et device_tag — c'est la **coquille produit locale** (voir § 0.3).

3.3 Backend — core/*, db/*, services/*, tasks/*

Figure 8 — c4-3-3-backend



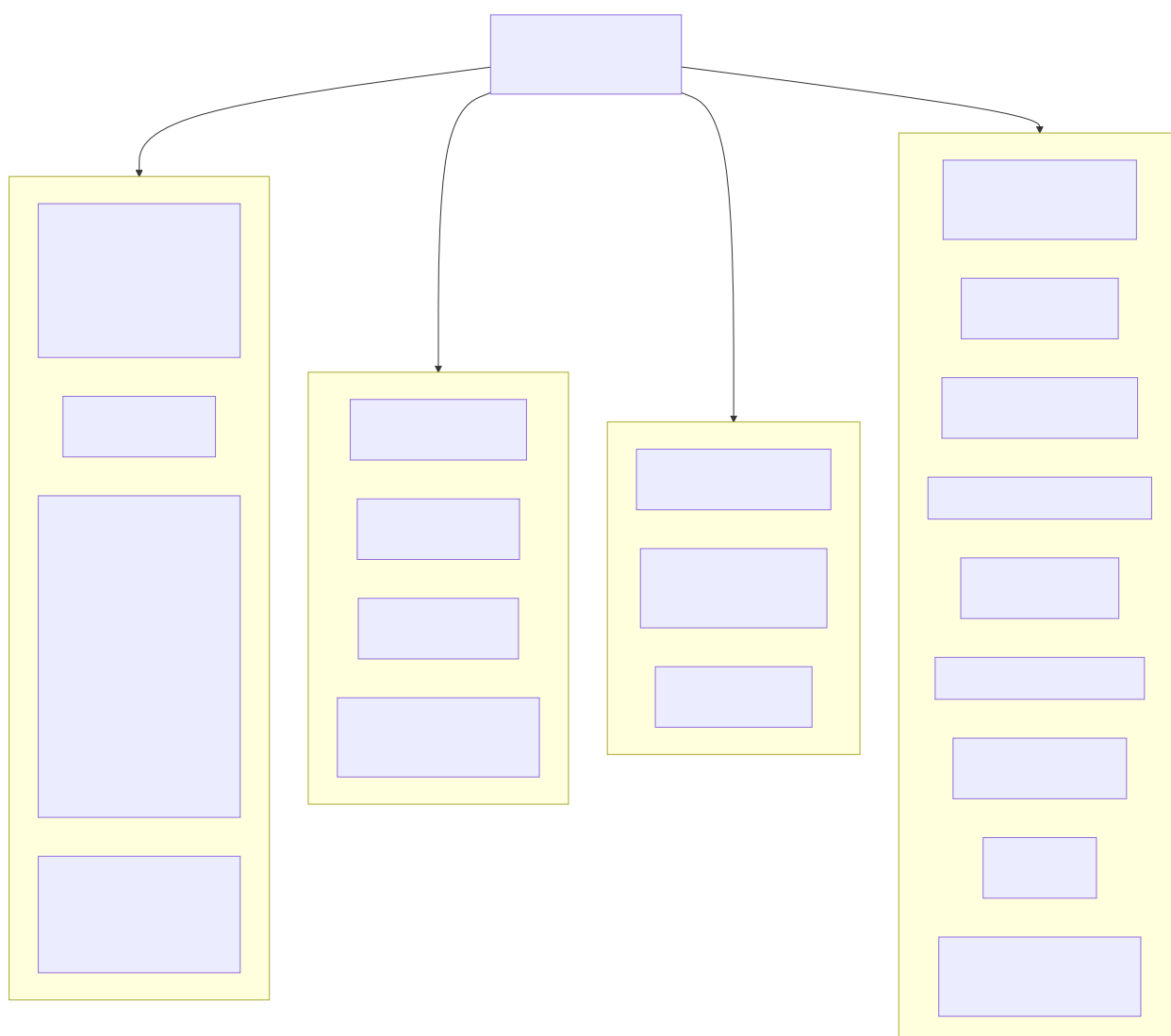
Lecture du diagramme. Les couches transverses sont strictement isolées par responsabilité :

- `core/*` : configuration, sécurité, Celery, Redis, rate limit, logs.
- `db/*` : engine SQLAlchemy async, base, seeds (uniquement en `ENVIRONMENT=development`).
- `services/*` : logique métier transverse (OCR, ISO 20022).
- `tasks/*` : workers Celery. Chaque module peut avoir son propre fichier `tasks.py` ; les tâches planifiées sont dans `tasks/maintenance.py`, `tasks/sanctions.py`, `tasks/kyc.py`. `tasks_demo.py` contient les helpers d'évidence (à ne pas traiter comme contrat métier).

Cross-link. Détail dans [docs/source-tree-analysis.md](#) § « Backend Tree ».

3.4 Mobile PWA — route map, contexts, services

Figure 9 — c4-3-4-mobile

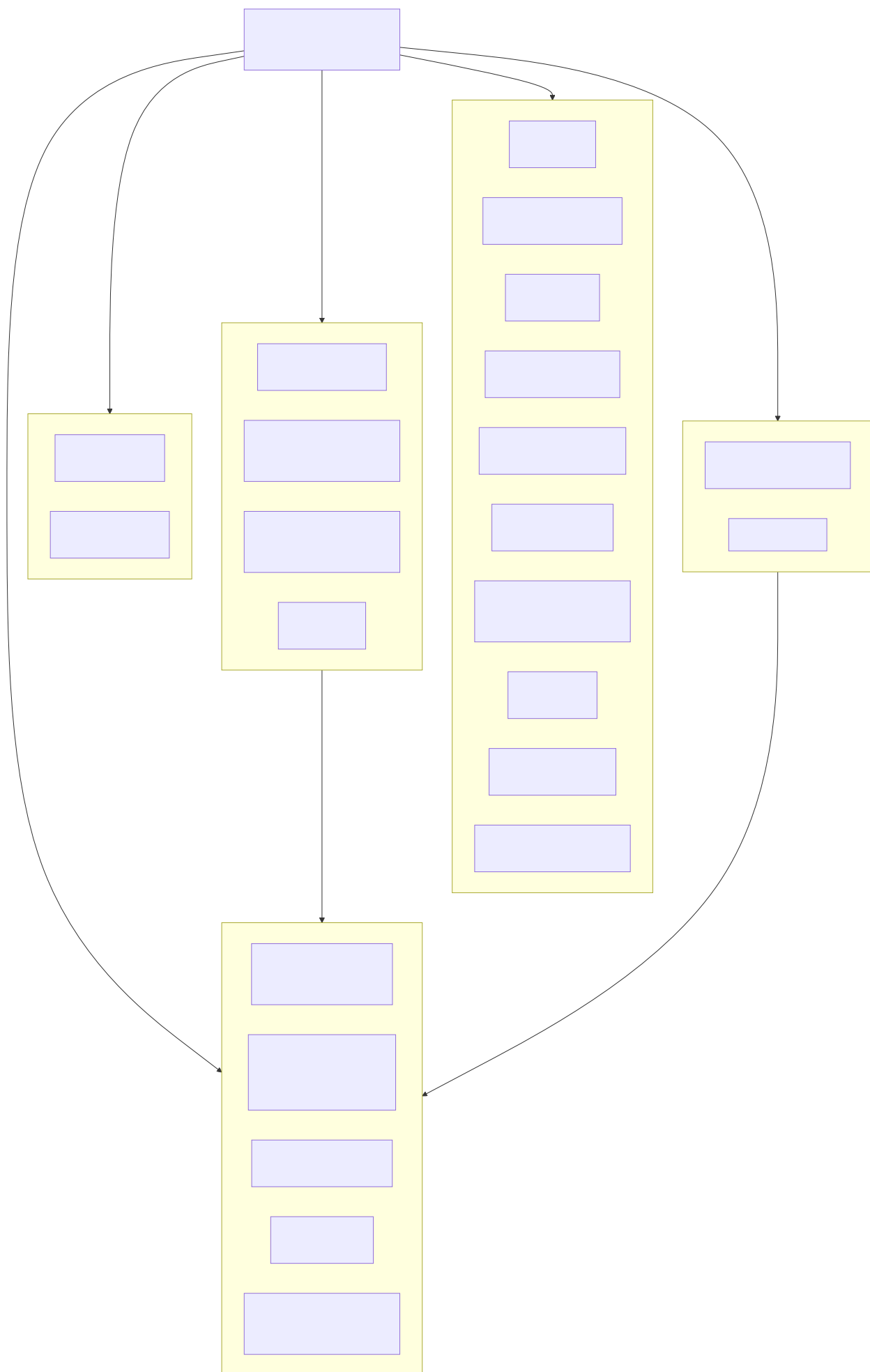


Lecture du diagramme. Le mobile suit un schéma **RouteMap → Providers → Guards → Contexts → Services → Views**. Les KYC views sont nombreuses (~25 écrans) et strictement ordonnées par useKycFlow. Le mode offline est supporté par IndexedDB (kycOfflineStore) et un service de replay (kycSyncService) qui empêche la soumission tant que la queue n'est pas vidée.

Cross-link. Détail dans [docs/component-inventory.md](#) § « Mobile PWA ».

3.5 Backoffice SPA — route map, pages, services

Figure 10 — c4-3-5-backoffice



Lecture du diagramme. Le backoffice est une SPA avec un **routeur protégé par rôle**. Les pages sont regroupées par rôle (validation pour Jean, compliance pour Thomas, command-center pour Sylvie, admin pour Admin IT) avec un partage de certains écrans (analytics est multi-rôles). Les services dossier-service.ts et aml-service.ts encapsulent les appels API par domaine.

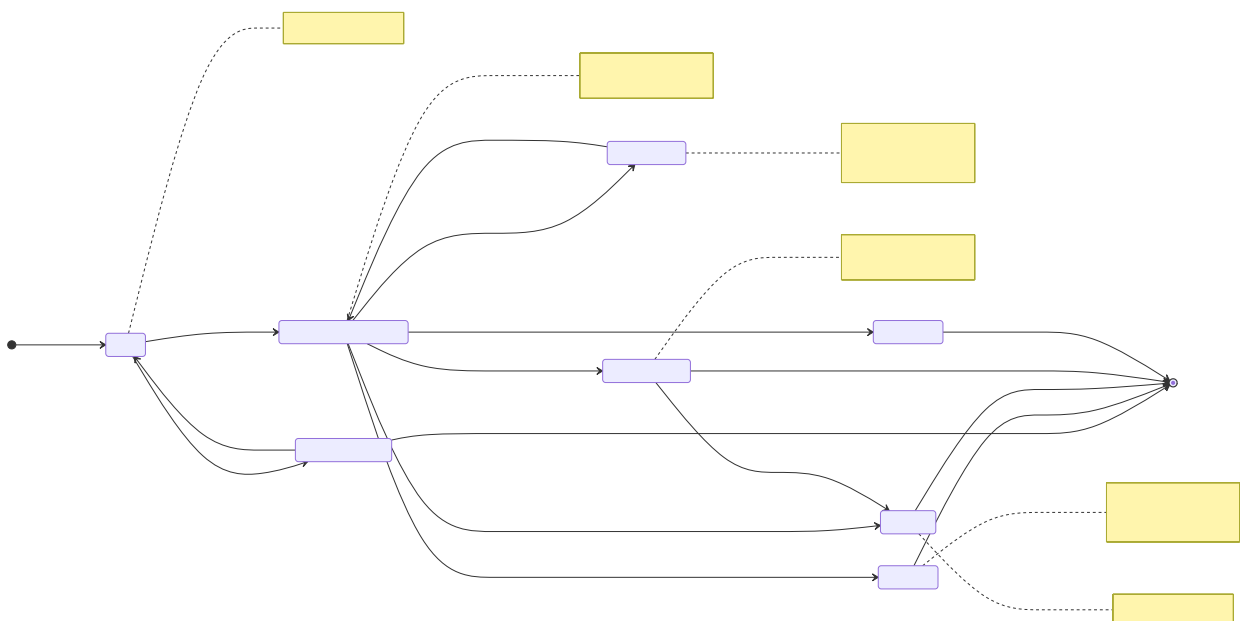
Cross-link. Détail dans [docs/component-inventory.md](#) § « Backoffice SPA » et [docs/api-contracts.md](#) § « Backoffice API ».

4. Niveau 4 – Code (C4 L4) – parcours critiques

Cette section présente les diagrammes au niveau du code pour les **parcours les plus critiques** : état KYC, soumission, OCR/biométrie, Celery, modèle de données, RBAC, handoff aval.

4.1 Machine d'état KYC et access_level

Figure 11 – c4-4-1-niveau



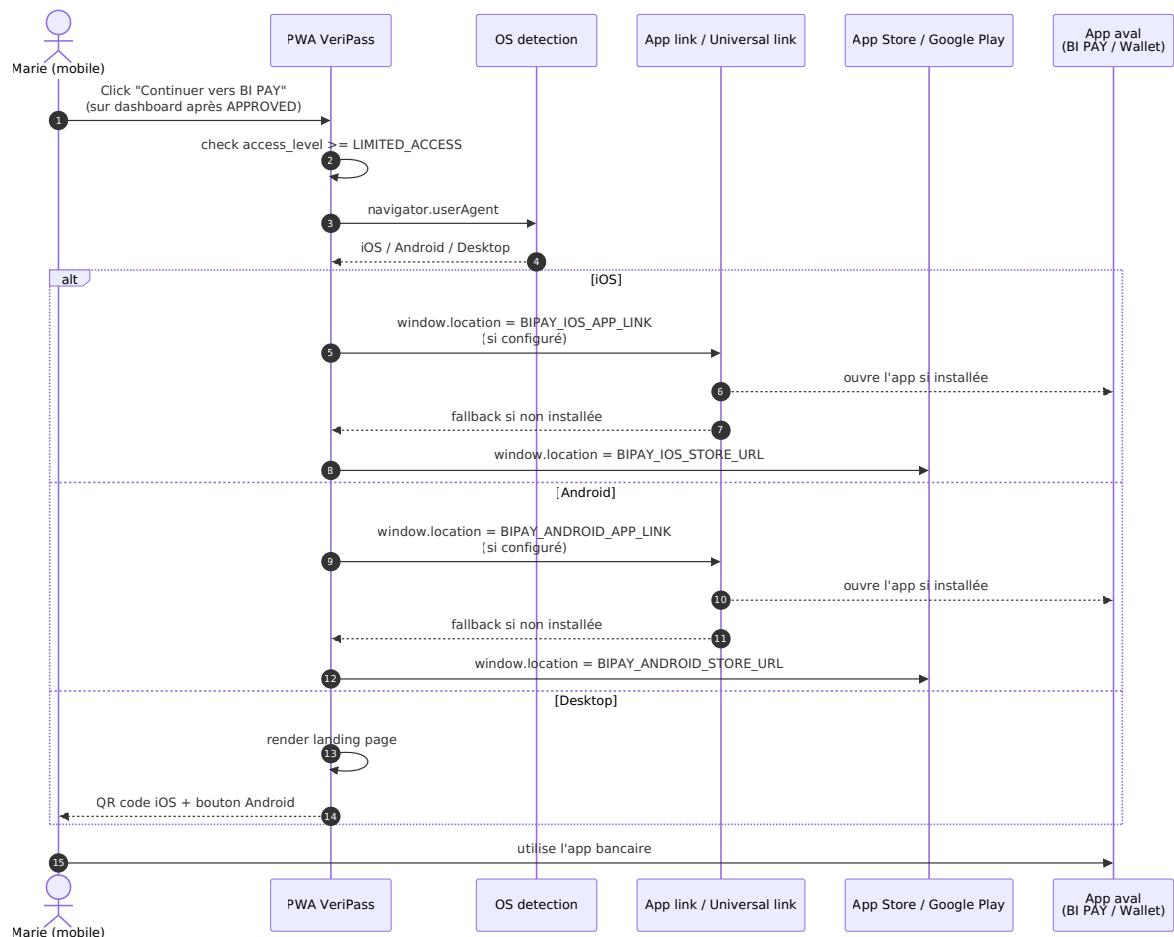
Lecture du diagramme. Le status (cycle de vie) et l'access_level (permission) sont **deux dimensions parallèles** :

status	access_level	par défaut Éditable ?
DRAFT	GUEST	oui
LOCKED_LIVENESS	RESTRICTED	non
PENDING_AGENT_REVIEW	RESTRICTED	non
PENDING_INFO	RESTRICTED	oui (resoumission possible)
APPROVED	LIMITED_ACCESS	non
FRAUD_SUSPECT	DISABLED	non
REJECTED	GUEST	non
ABANDONED	GUEST	non

Source de vérité : code/backend/app/modules/kyc/schemas.py (énumérations KYCStatus, AccessLevel) et code/backend/app/modules/kyc/service.py (transitions). Voir [docs/data-models.md](#) § « KYC Tables ».

4.2 Séquence du parcours KYC complet

Figure 12 — c4-4-10-handoff



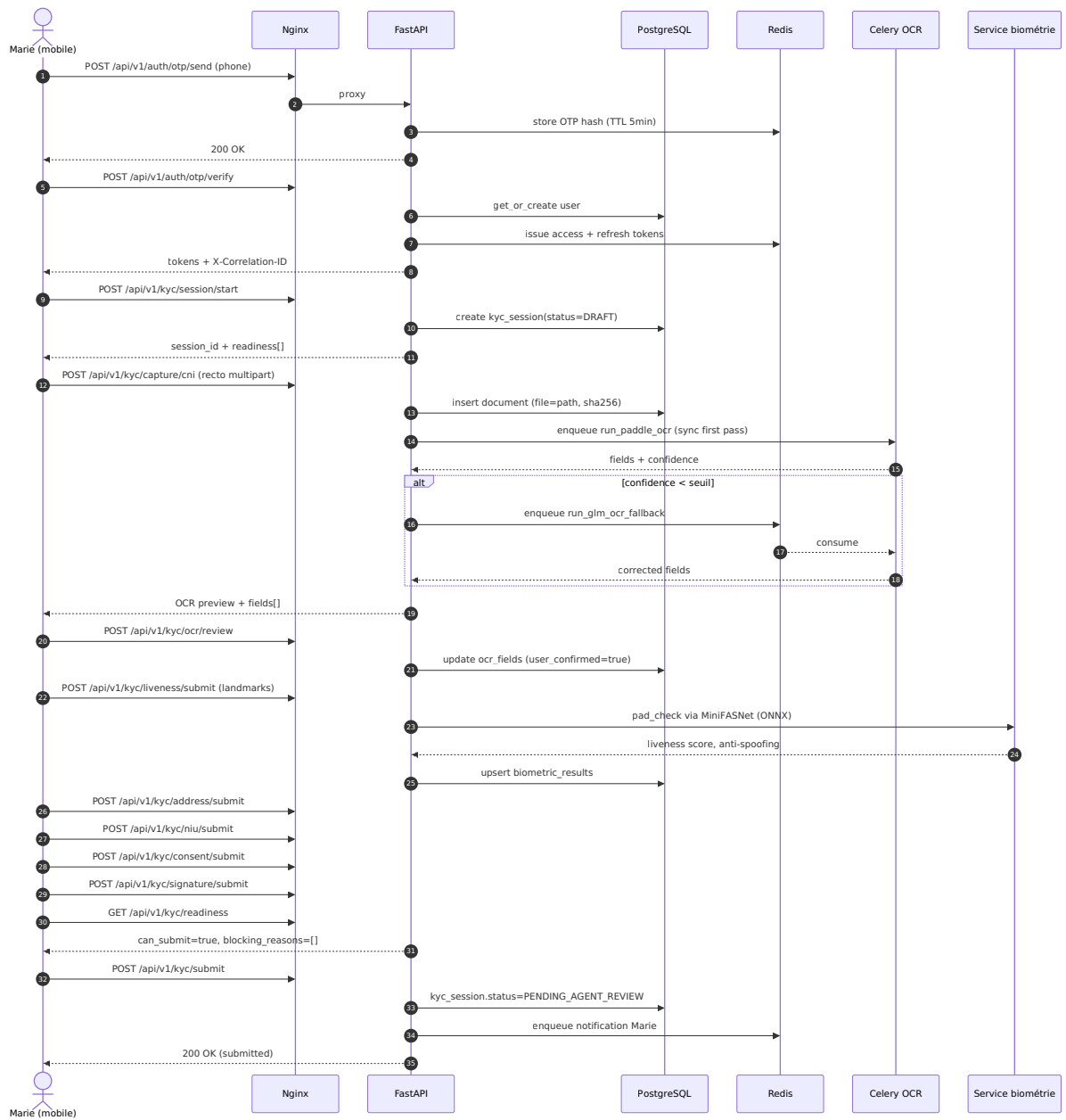
Lecture du diagramme. Le parcours complet comporte ~15 appels. Trois points d'attention :

1. **PaddleOCR synchrone** au premier passage ; **GLM-OCR asynchrone** en fallback.
2. **MiniFASNet** est invoqué côté serveur (sur le SELFIE capturé), pas côté client.
3. **/kyc/readiness** est un GET idempotent qui calcule l'état de complétude avant la soumission.

Cross-link. Détail dans [docs/architecture.md](#) § « KYC Submission Flow » et [docs/api-contracts.md](#) § « KYC API ».

4.3 Pipeline OCR et biométrie

Figure 13 — c4-4-2-séquence

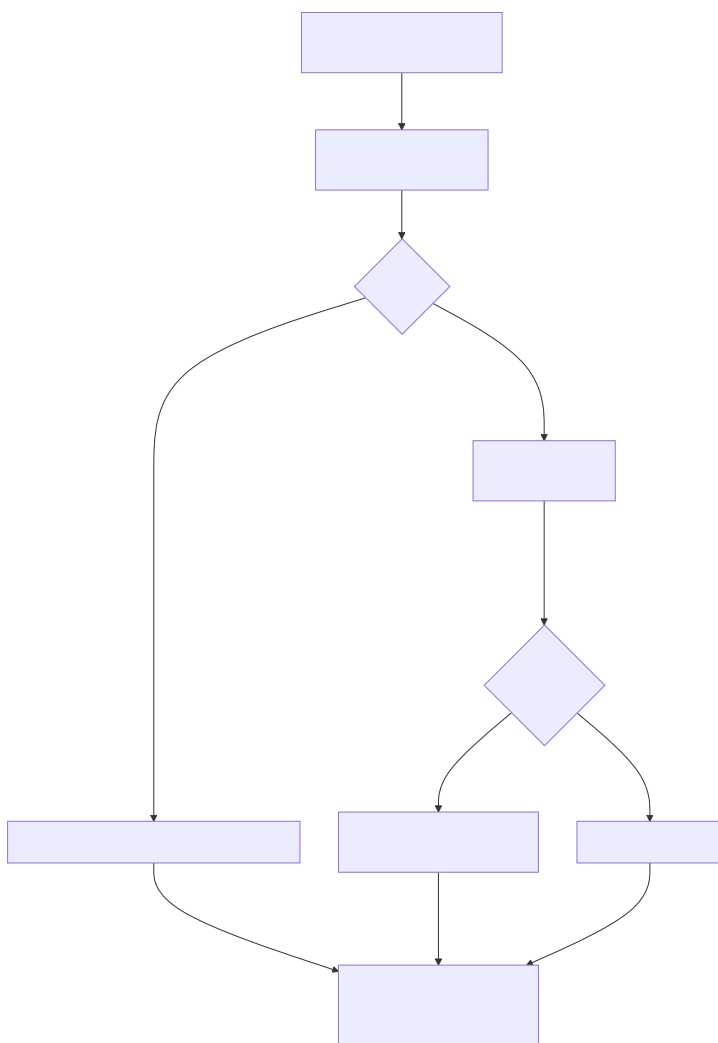
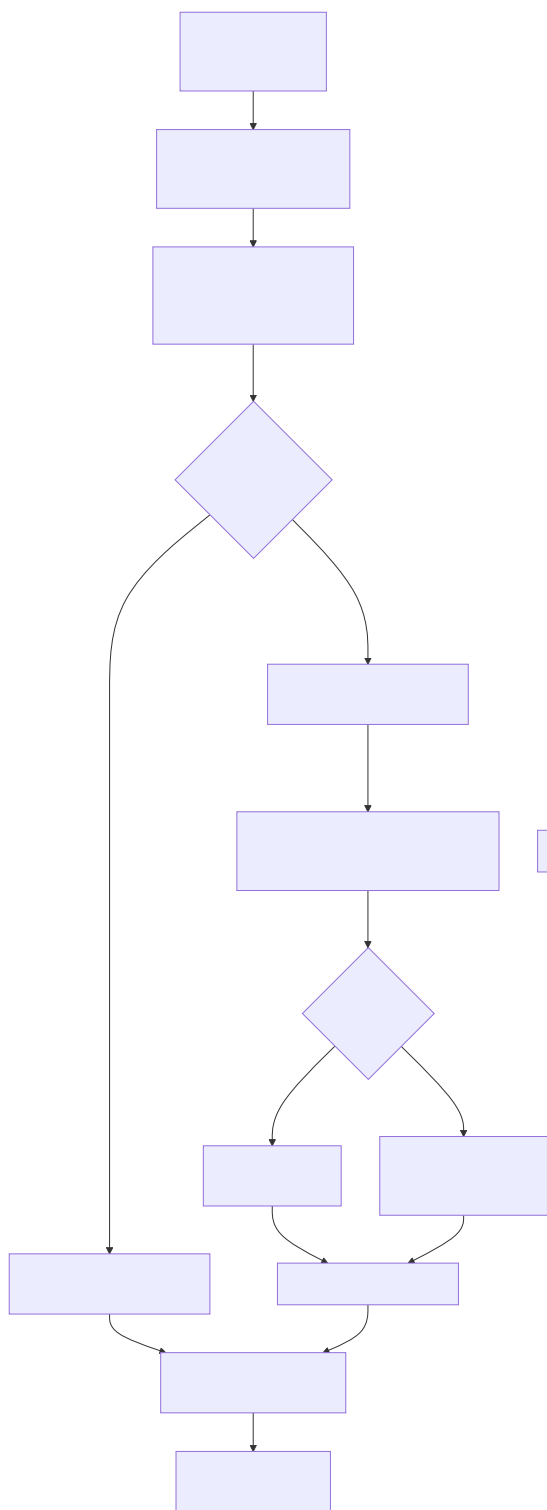


Lecture du diagramme. Le pipeline OCR est **sync pour PaddleOCR, async pour GLM-OCR**. Le pipeline biométrique est **sync et local** (MiniFASNet sur l'image SELFIE déjà stockée, landmarks fournis par le mobile). Les modèles sont montés depuis `OFFLINE_MODELS_DIR` en mode souverain. Le SHA-256 du modèle MiniFASNet est validé contre `MINIFASNET_MODEL_SHA256` quand fourni.

Cross-link. Détail dans [docs/architecture.md](#) § « OCR And Biometrics ».

4.4 Topologie Celery et beat schedule

Figure 14 — c4-4-3-pipeline



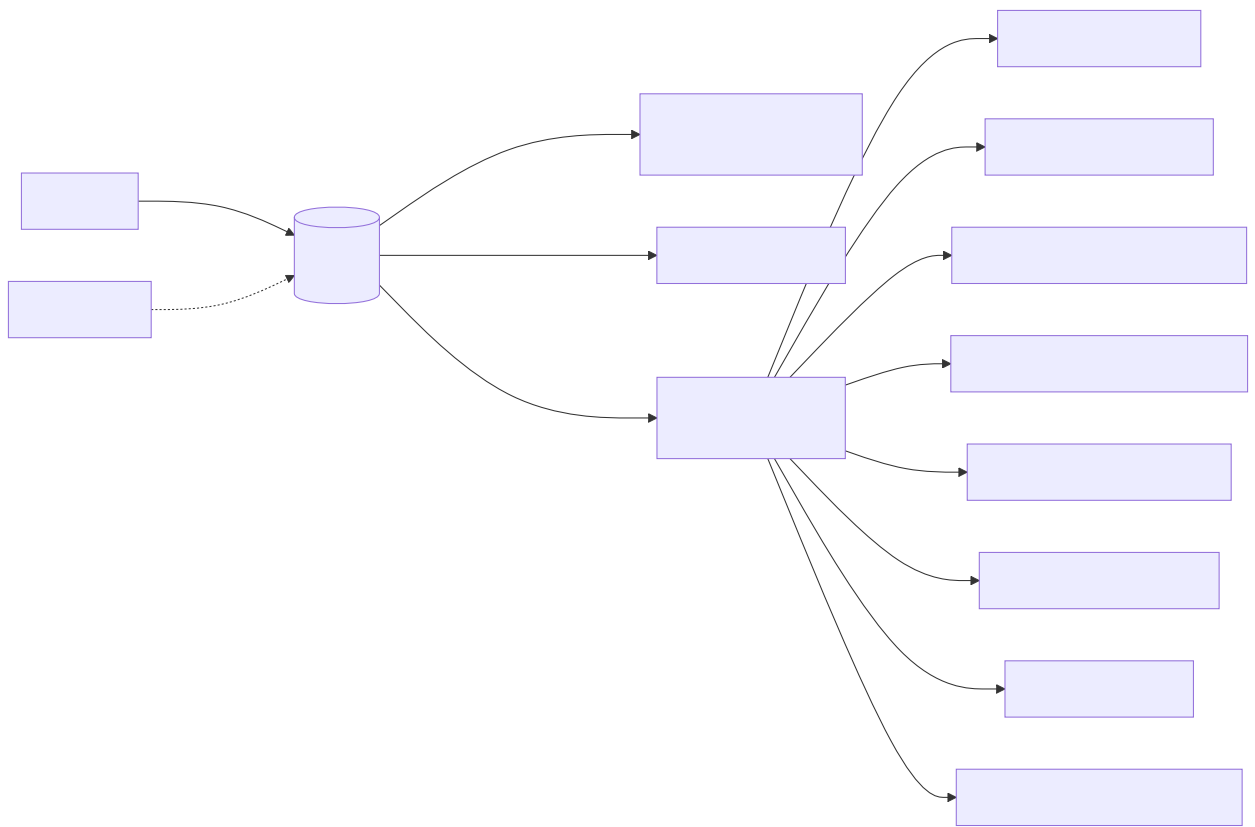
Beat schedule (extrait) :

Job	Schedule (UTC)	Implémentation	Notes
backup-db-daily	01:00 quotidien	tasks/maintenance.py:backup_postgres	pg_dump vers code_db_backups ; gated par ENABLE_BACKUPS=true
backup-images-weekly	Dimanche 02:00	tasks/maintenance.py:backup_kyc_images	Archive chiffrée des code_documents_storage nécessite BACKUP_ENCRYPTION_KEY
check-backup-catchup-daily	03:30 quotidien	tasks/ maintenance.py:check_and_catchup_backup	Déclenche un backup images si pas récent
detect-abandoned-sessions	03:00 quotidien	tasks/kyc.py	Marque les sessions KYC abandonnées
check-document-expiry-daily	03:20 quotidien	tasks/kyc.py	Signale les documents expirant
screen-active-clients-daily	04:30 quotidien	tasks/kyc.py	Re-screening sanctions d clients actifs
sync-sanctions-weekly	Lundi 02:00	tasks/sanctions.py	Refresh OpenSanctions
check-sanctions-staleness	04:00 quotidien	tasks/sanctions.py	Sync si données plus vieilles que le seuil

Cross-link. Détail dans [docs/architecture.md](#) § « Async Jobs ».

4.5 Modèle de données consolidé

Figure 15 — c4-4-4-topologie

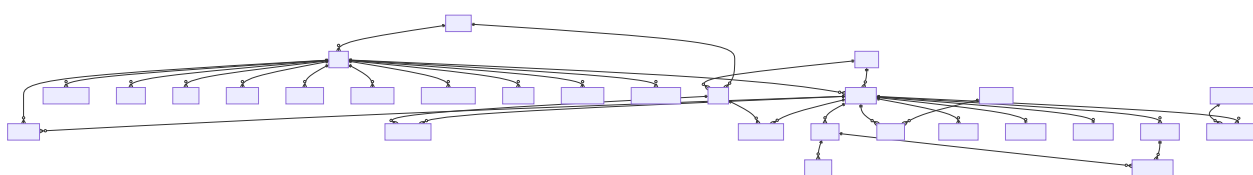


Lecture du diagramme. Le modèle de données est centré sur KYC_SESSIONS. Les tables BANK_CARDS, TRANSFERS, TRANSACTIONS, SAVINGS_POCKETS appartiennent au **module banking local** (coquille produit, pas un adaptateur core banking — voir § 0.3). PROVISIONING_BATCHES est une table interne d'administration ; son nom évoque le provisioning mais **elle n'est pas une intégration DGI, Sopra, ou core banking** ; c'est un simple suivi de batchs administratifs.

Cross-link. Détail dans [docs/data-models.md](#).

4.6 Séquence de la revue backoffice

Figure 16 — c4-4-5-modèle

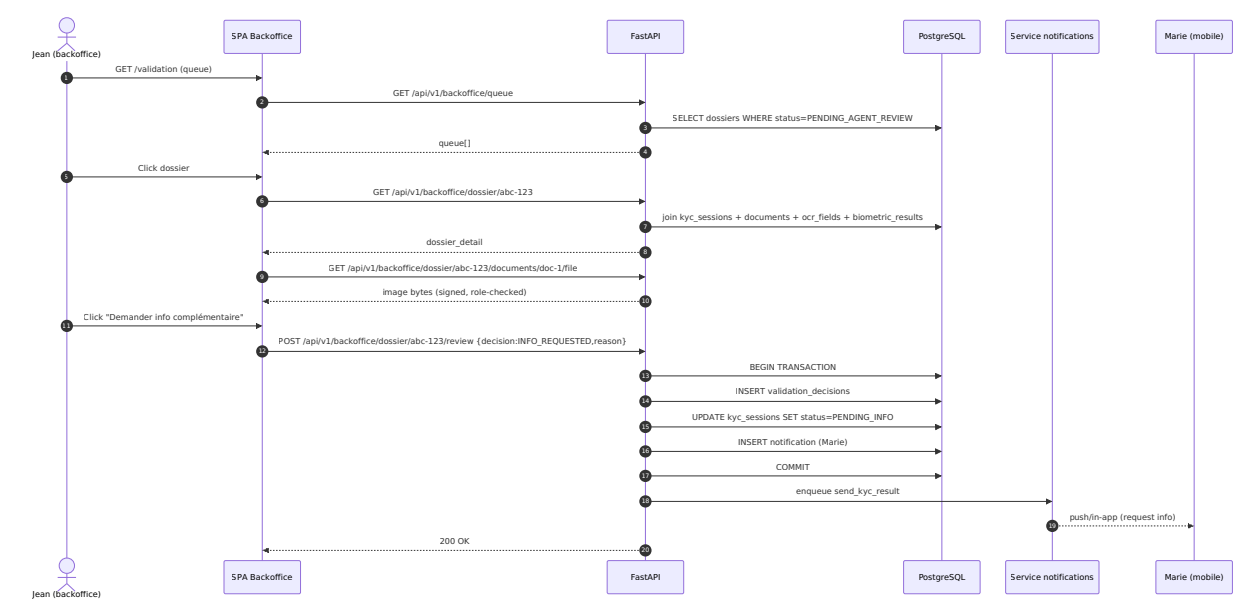


Lecture du diagramme. Toutes les décisions sont **transactionnelles et traçables** (table validation_decisions append-only, audit_log mis à jour). La notification à Marie est asynchrone. Les fichiers sont servis par un endpoint dédié qui re-vérifie le rôle de l’agent.

Cross-link. Détail dans [docs/api-contracts.md](#) § « Backoffice API ».

4.7 Cycle de vie d’un dossier (vue gitGraph)

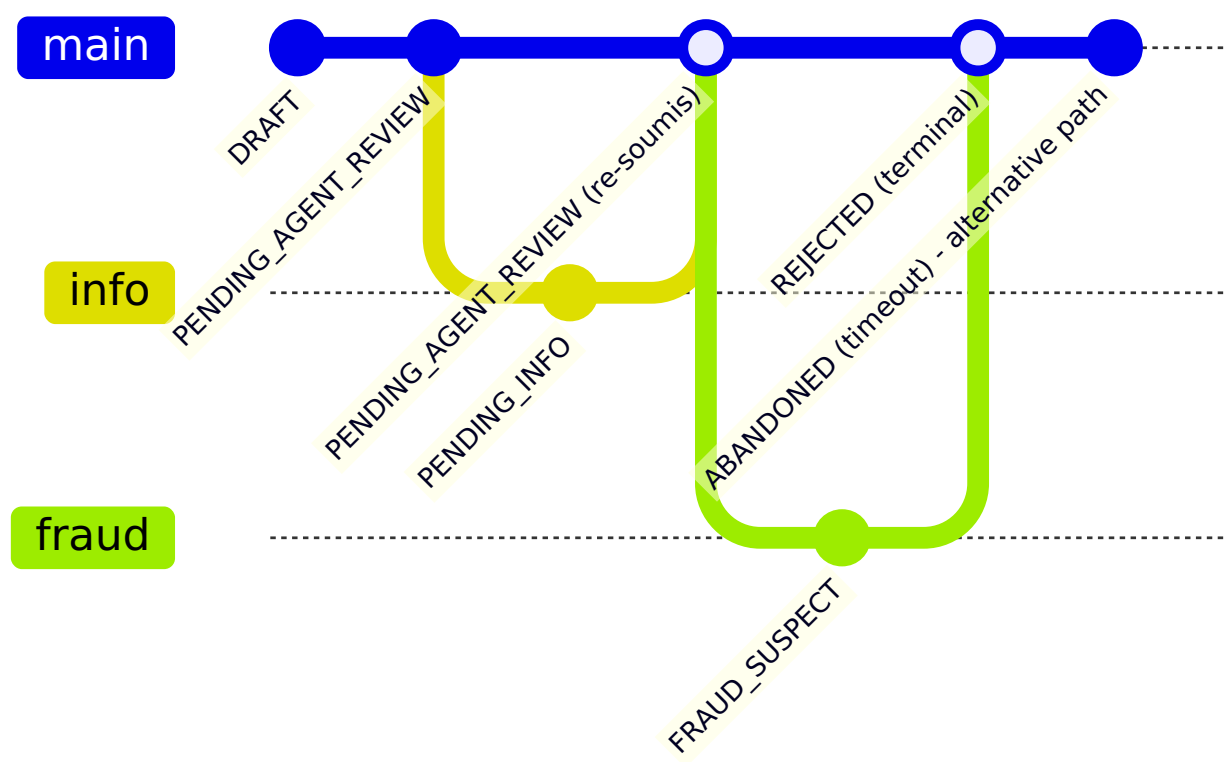
Figure 17 — c4-4-6-séquence



Lecture du diagramme. Cette vue gitGraph modélise les **branches de cycle de vie** d'un dossier. La branche **info** représente un aller-retour **PENDING_INFO** → **PENDING_AGENT_REVIEW** ; la branche **fraud** représente un cas **FRAUD_SUSPECT** → **REJECTED** ; le chemin direct **ABANDONED** est un chemin terminal alternatif. Le commit final valide le **APPROVED** → **LIMITED_ACCESS**.

4.8 Matrice RBAC

Figure 18 — c4-4-7-cycle



Lecture du diagramme. Le RBAC est appliqué à **deux endroits** :

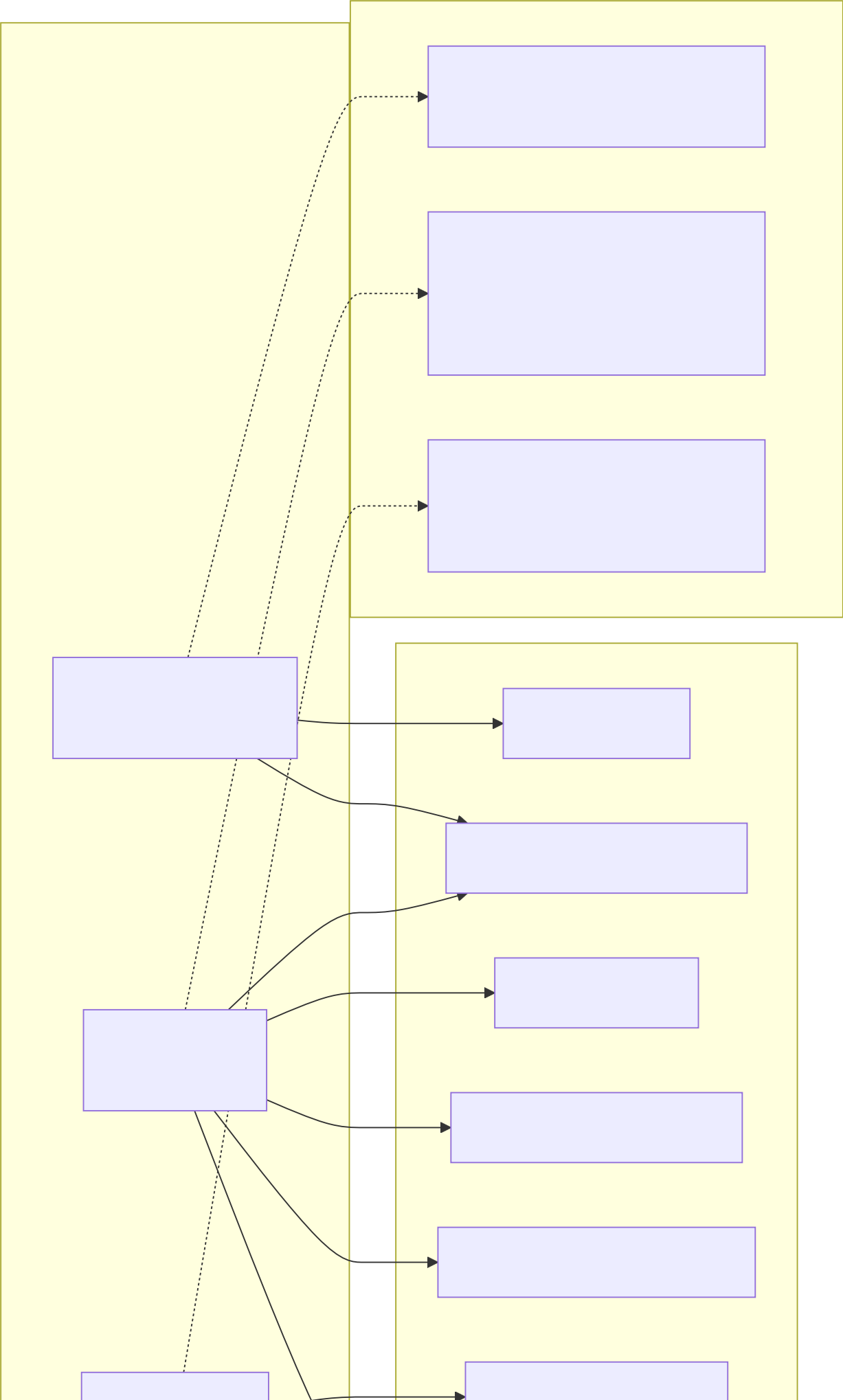
1. **Côté backoffice** (routeur React) : `ProtectedRoute` + `RoleRedirect` filtrent l'accès aux pages.
2. **Côté backend** (FastAPI dependencies) : `get_current_agent` + `require_agent_role(...)` filtrent l'accès aux endpoints ET aux décisions.

Règle d'or : toute modification de rôle doit être faite **simultanément** des deux côtés, et un test d'accès non-autorisé doit être ajouté.

Cross-link. Détail dans [docs/api-contracts.md](#) § « Decision rules in `submit_review_decision` ».

4.9 Happy-path end-to-end

Figure 19 — c4-4-8-matrice

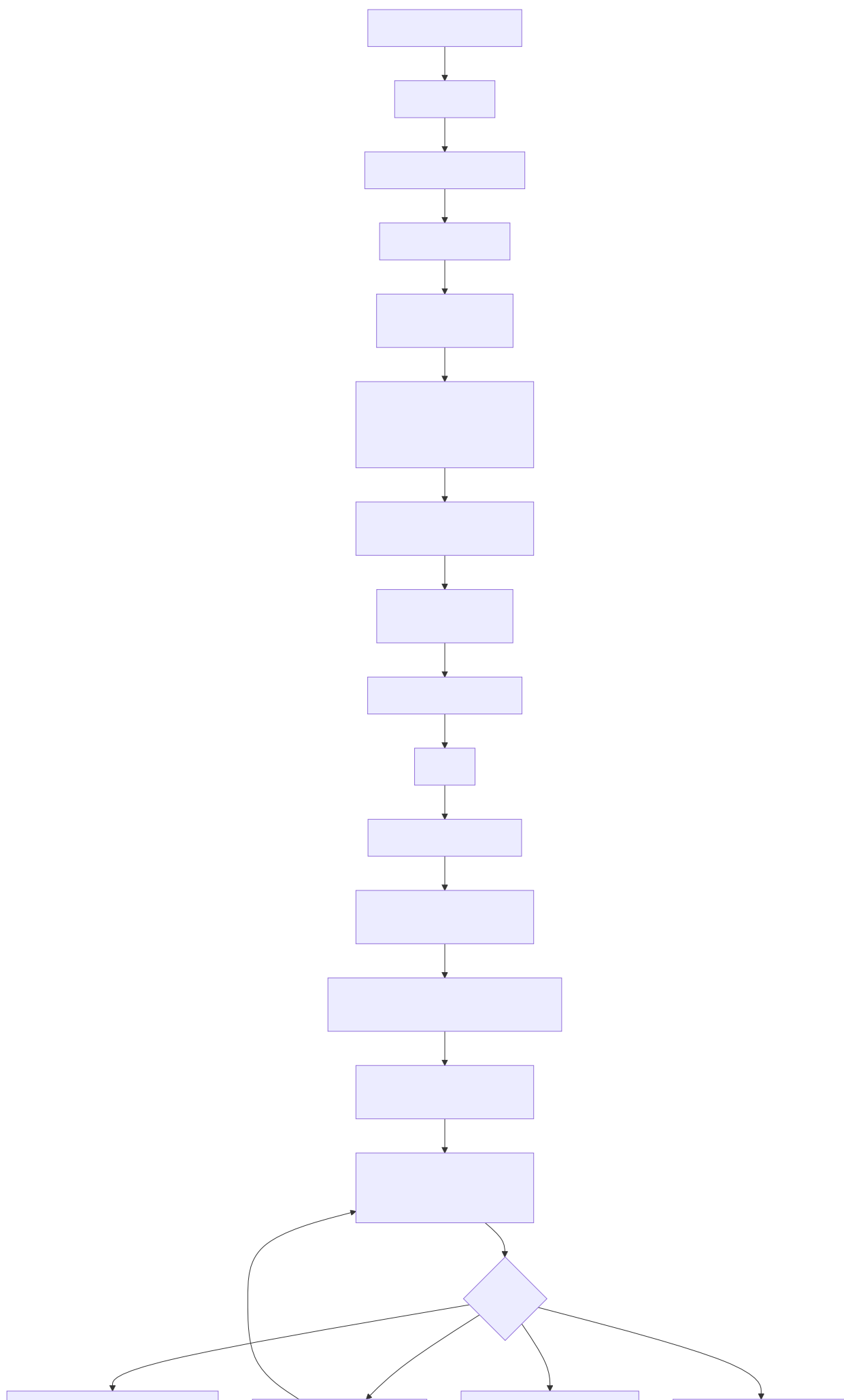


Lecture du diagramme. Le happy-path est linéaire avec un seul aller-retour possible (PENDING_INFO). L'approbation débloque l'access_level LIMITED_ACCESS et déclenche le handoff OS-aware vers les apps aval. Les chemins REJECTED et FRAUD_SUSPECT sont terminaux.

Cross-link. Détail dans [docs/maintainer-handbook.md](#) § « Main Workflows ».

4.10 Handoff mobile aval (app-link)

Figure 20 — c4-4-9-happy-path



Lecture du diagramme. Le handoff est **uniquement côté mobile**, basé sur des **variables d’environnement** (jamais de chaîne hard-codée) :

Variable	Usage
BIPAY_IOS_STORE_URL	App Store fallback iOS pour BI PAY
BIPAY_ANDROID_STORE_URL	Google Play fallback Android pour BI PAY
BIPAY_APP_LINK	Universal/app link optionnel iOS+Android
BICEC_WALLET_IOS_STORE_URL	App Store fallback iOS pour Wallet / Mobile-Banking
BICEC_WALLET_ANDROID_STORE_URL	Google Play fallback Android pour Wallet / Mobile-Banking
BICEC_WALLET_APP_LINK	Universal/app link optionnel iOS+Android

Valeurs publiques de référence (à confirmer avec BICEC avant release) :

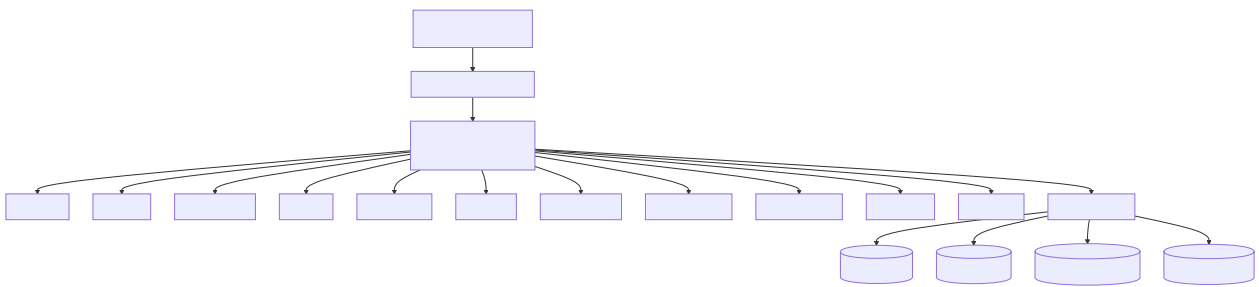
Destination	iOS	Android
BI PAY	https://apps.apple.com/ us/app/bipay-bicec- mobile-wallet/ id1532756992	https:// play.google.com/store/ apps/details? id=com.bicec.bipay
BICEC Mobile-Banking	https://apps.apple.com/ us/app/bicec-mobile- banking/id1011209991	(à confirmer)

Cross-link. Détail dans [docs/api-contracts.md](#) § « Downstream App Handoff ».

5. Opérations et handover

5.1 Topologie de déploiement

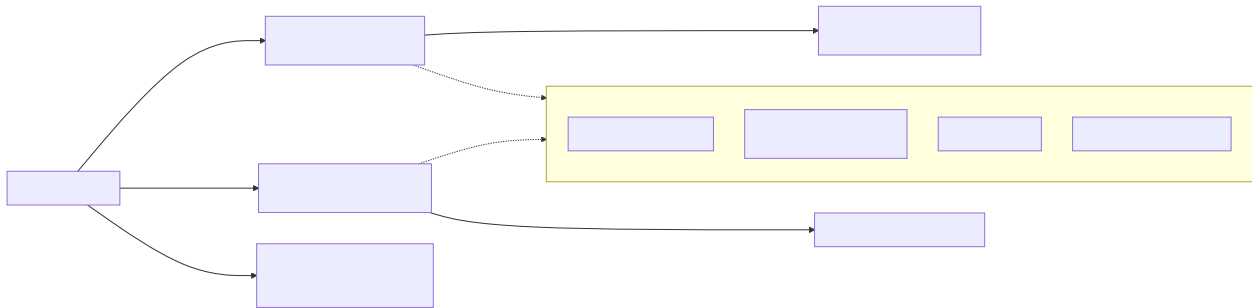
Figure 21 — c4-5-1-opérations



Cross-link. [docs/deployment-guide.md](#).

5.2 Topologie backup et rétention

Figure 22 — c4-5-2-topologie



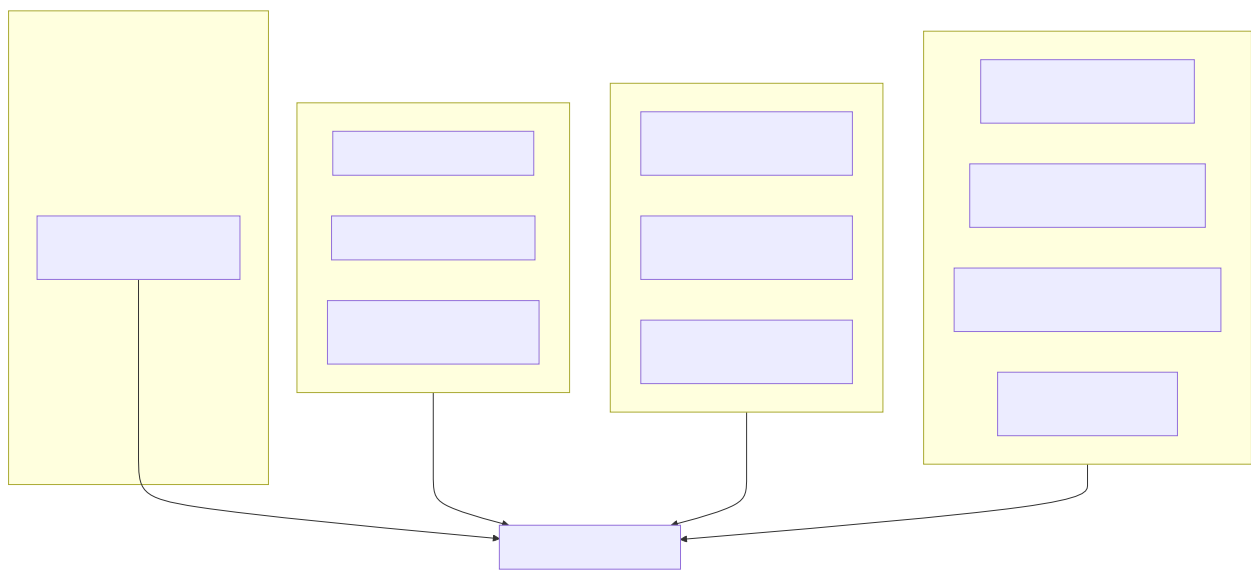
Rétention :

Donnée	Localisation	Backup	Rétention
PostgreSQL	code_db_storage	code_db_backups (/backups/db)	Selon retention-policy.md
KYC documents	code_documents_storage	archive chiffrée locale	Selon retention-policy.md
Modèles OCR/IA	code_models_storage (bind mount)	image / archive	Hors ligne ; pas un backup applicatif
Redis	redis_data	aucun (transient)	n/a

Cross-link. [docs/retention-policy.md](#) et [docs/deployment-guide.md](#) § « Backups ».

5.3 Carte d’observabilité

Figure 23 — c4-5-3-carte



Cross-link. [docs/operations-runbook.md](https://docs.operations-runbook.md).

5.4 Où changer quoi

Demande de changement	Fichiers principaux	Vérifier aussi
Ajouter un écran mobile KYC	code/mobile/src/App.tsx, code/mobile/src/views/kyc/*, code/mobile/src/hooks/useKycFlow.tsx, code/mobile/src/contextts/KycContext.tsx	backend modules/kyc/router.py, tests E2E mobile
Ajouter un appel API mobile	code/mobile/src/services/apiClient.ts (ou service de feature)	router backend, types TS, tests
Modifier les champs OCR	code/backend/app/modules/kyc/service.py, code/backend/app/services/ocr_service.py	écrans OCR mobile et backoffice
Modifier le scoring liveness	code/backend/app/modules/kyc/service.py	code/mobile/src/views/kyc/LivenessScreen.tsx
Modifier la readiness de soumission	code/backend/app/modules/kyc/router.py	code/mobile/src/views/kyc/ReviewScreen.tsx
Modifier la queue backoffice	code/backend/app/modules/backoffice/router.py	code/backoffice/src/services/dossier-service.ts, pages validation
Modifier AML	code/backend/app/modules/aml/service.py, code/backend/app/modules/aml/router.py	pages compliance
Modifier analytics	code/backend/app/modules/analytics/service.py	code/backoffice/src/pages/analytics/AnalyticsPage.tsx
Ajouter une table	modèle SQLAlchemy + migration Alembic + schemas Pydantic + tests + docs/data-models.md	—
Modifier une config déploiement	code/docker-compose.yml, code/.env.example, code/infra/nginx/nginx.conf	—

Demande de changement	Fichiers principaux	Vérifier aussi
Modifier un job planifié	code/backend/app/core/ celery_config.py (beat), implémentation tâche, worker Compose	idempotence de la tâche
Modifier un rôle	require_agent_role(...) backend + routes backoffice + ProtectedRoute + tests 401/403	—

5.5 Invariants critiques

1. **Ne jamais supprimer les volumes Docker persistants** (code_db_storage, code_documents_storage, code_db_backups) comme fix ponctuel.
2. **Les fichiers KYC** vivent dans le volume de documents ; PostgreSQL ne stocke que les chemins et les SHA-256.
3. **status (KYC) et access_level (permission) sont deux dimensions orthogonales** — ne jamais les fusionner.
4. **APPROVED mappe par défaut sur LIMITED_ACCESS.**
5. **FRAUD_SUSPECT mappe sur DISABLED.**
6. **PENDING_INFO reste éditabile** côté client (le client doit pouvoir uploader les pièces demandées).
7. **Toute vérification de rôle agent** doit être faite côté backend ET reflétée côté frontend (routes + ProtectedRoute).
8. **Le X-Device-Tag est obligatoire** sur les actions sensibles dès qu'un user a au moins un device actif enregistré.
9. **OTP_MODE=dev_local est strictement interdit en production** (rejeté au démarrage par core/config.py).
10. **OCR_ONLINE=false** est la valeur normale en mode souverain / démo offline.
11. **VeriPass n'est PAS** une intégration DGI, Sopra Amplitude, ou core banking. Tout document historique qui le dit est obsolète sur ce point.
12. **Le handoff vers BI PAY / Wallet / Mobile-Banking** se fait par **app-link + fallback store**, jamais par intégration backend.
13. **Le openapi-spec.json statique** peut être périmé ; le runtime /api/v1/openapi.json et /api/v1/docs font foi.

5.6 Commandes smoke (copier-coller)

Pré-requis : Docker Desktop (WSL2 activé) et PowerShell sur Windows, ou Docker + bash sur Linux.

```
# 1. Copier la config et créer les volumes
copy code\.env.example code\.env
```

```
notepad code\.env  
.\code\scripts\ensure_docker_volumes.ps1
```

2. Construire et démarrer la stack MVP

```
docker compose -f code/docker-compose.yml build  
docker compose -f code/docker-compose.yml up -d api pwa backoffice  
postgres redis celery_ocr celery_notifications celery_beat mailpit  
nginx flower
```

3. Vérifier l'état

```
docker compose -f code/docker-compose.yml ps
```

4. Smoke test

```
Invoke-RestMethod http://localhost:8001/api/health  
.\code\scripts\smoke_mvp_docker.ps1
```

5. Migrations

```
docker compose -f code/docker-compose.yml exec -T api alembic current  
docker compose -f code/docker-compose.yml exec -T api alembic upgrade  
head
```

6. Backend tests

```
docker compose -f code/docker-compose.yml exec -T api pytest
```

7. Mobile (hors stack)

```
cd code\mobile  
bun install  
bun run test  
bun run build
```

8. Backoffice (hors stack)

```
cd code\backoffice  
bun install  
bun run test  
bunx tsc --noEmit  
bun run build
```

9. URLs manuelles à vérifier

```
# - https://localhost/mobile/  
# - https://localhost/back-office/  
# - https://localhost/api/v1/docs  
# - https://localhost/flower/  
# - http://localhost:8025/ (Mailpit)
```

5.7 Zones à risque connu

Zone	Pourquoi c'est risqué	Comment travailler prudemment
Warmup OCR	Modèles lourds, premier appel, pression mémoire	Vérifier logs, montages modèles, mémoire WSL2, isoler les changements
Readiness KYC	Beaucoup de champs, conditions offline	Tester sessions partielles + happy-path complet
RBAC backoffice	Double enforcement front/back	Mettre à jour les deux, tester les chemins non autorisés
Migrations Alembic	Volumes persistants en production	Backup avant toute migration risquée
Pathing Nginx	/mobile/, /back-office/, deep-links	Tester le refresh direct sur routes imbriquées
Celery beat	Backups, pruning, sanctions, expiry	Vérifier le schedule et l'idempotence des tâches
Docker socket (celery_beat)	Puissant, dangereux en production	Restreindre ou redéfinir pour la production
Fichiers evidence	Volumineux, générés	Régénérer plutôt qu'éditer à la main
Modèles hors-ligne	Gigas, montage obligatoire	Garder OFFLINE_MODELS_DIR synchronisé entre dev/prod
BACKUP_ENCRYPTION_KEY	Ne doit jamais être JWT_SECRET	Forcer valeur distincte en CI

5.8 Glossaire FR/EN

FR	EN	Signification
KYC	KYC	Know Your Customer — identification client
PWA	PWA	Progressive Web App
SPA	SPA	Single Page Application
OCR	OCR	Reconnaissance optique de caractères
PAD	PAD	Presentation Attack Detection (anti-spoofing)
PEP	PEP	Personnes Exposées Politiquement
AML/CFT	AML/CFT	

FR	EN	Signification
		Anti-Money Laundering / Counter Financing of Terrorism
COBAC	COBAC	Commission Bancaire d'Afrique Centrale (régulateur)
NIU	NIU	Numéro d'Identification Unique (fiscal Cameroun)
CNI	CNI	Carte Nationale d'Identité
RGPD / CGU	GDPR / ToS	Conformité et conditions d'usage
App-link	App link / Universal link	Schéma d'URL profond vers une app native
DGI	DGI	Direction Générale des Impôts (hors périmètre)
Sopra Amplitude	Sopra Amplitude	Core banking éditeur (hors périmètre)
Beat	Beat	Scheduler Celery
Handoff	Handoff	Transfert OS-aware vers une autre app

5.9 Versioning et changelog

- **Version du document** : 1.0.0 (2026-06-02).
- **Politique** : tout changement de portée, d'architecture majeure, ou d'invariant doit être reflété ici et dans docs/documentation-authority.md.
- **Render** : régénérer les SVG et le PDF via docs/c4-architecture/render.ps1 (Windows) ou docs/c4-architecture/render.sh (Linux/WSL). Voir [Annexe B](#).
- **Changelog** :

Date	Version	Auteur	Note
2026-06-02	1.0.0	Ken	Création du document maître C4, re-cadrage DGI/ Amplitude, handoff mobile aval explicite.

6. Annexes

Annexe A — Index croisé master ↔ docs détaillées

Section master	Doc de référence	Diagrammes liés
§ 0 Cadrage	docs/project-overview.md , docs/documentation-authority.md	—
§ 1 Contexte	docs/project-overview.md § « Main Users »	docs/diagrams/use-case-diagrams.md , docs/diagrams/user-personas.md , docs/diagrams/user-journey-maps.md
§ 2 Containers	docs/architecture.md § « Container Architecture »	—
§ 2.3 Nginx	docs/architecture.md § « Nginx Routing »	—
§ 3.1 Backend entry	docs/architecture.md § « Backend Architecture »	—
§ 3.2 Modules	docs/component-inventory.md	docs/diagrams/architecture/sitemap-diagram.md
§ 3.3 Transverse	docs/source-tree-analysis.md § « Backend Tree »	—
§ 3.4 Mobile	docs/component-inventory.md § « Mobile PWA »	docs/diagrams/flows/*
§ 3.5 Backoffice	docs/component-inventory.md § « Backoffice SPA »	—
§ 4.1 État KYC	docs/data-models.md § « KYC Tables »	docs/diagrams/statechart-access.md (re-authored)
§ 4.2 Séquence KYC	docs/architecture.md § « KYC Submission Flow »	docs/diagrams/sequence-capture-upload.md (re-authored)
§ 4.3 OCR/biométrie	docs/architecture.md § « OCR And Biometrics »	—
§ 4.4 Celery		—

Section master	Doc de référence	Diagrammes liés
	docs/architecture.md § « Async Jobs »	
§ 4.5 Modèle	docs/data-models.md	docs/diagrams/er-application.md (re-authored)
§ 4.6 Revue BO	docs/api-contracts.md § « Backoffice API »	—
§ 4.8 RBAC	docs/api-contracts.md	—
§ 4.10 Handoff	docs/api-contracts.md § « Downstream App Handoff »	—
§ 5 Opérations	docs/operations-runbook.md , docs/deployment-guide.md	docs/diagrams/information-architecture.md (re-authored)
§ 5.4 Change map	docs/maintainer-handbook.md § « Where To Make Changes »	—
§ 5.5 Invariants	docs/maintainer-handbook.md § « Critical Invariants »	—
§ 5.6 Smoke	docs/development-guide.md , docs/operations-runbook.md	—

Annexe B — Rendu SVG et PDF — comment régénérer

Les diagrammes de ce document sont en syntaxe **Mermaid** et sont rendus nativement par GitHub, VS Code, Obsidian, etc. Pour des exports SVG / PDF :

Pré-requis :

- Node.js 18+
- `npm i -g @mermaid-js/mermaid-cli` (binaire mmdc)
- (optionnel, pour le PDF) Pandoc + un moteur LaTeX (lualatex / tectonic) ou WeasyPrint

Windows (PowerShell) :

```
cd docs\c4-architecture
.\render.ps1           # SVG uniquement
.\render.ps1 -Pdf      # SVG + PDF
```

Linux / WSL :

```
cd docs/c4-architecture
./render.sh                # SVG uniquement
./render.sh --pdf          # SVG + PDF
```

Les fichiers produits :

- docs/c4-architecture/diagrams/c4-{level}-{num}-{slug}.svg — un SVG par diagramme.
- docs/c4-architecture/BICEC-VERIPASS-VUE-ENSEMBLE.pdf — PDF imprimable A4.
- docs/c4-architecture/mermaid/*.mmd — sources Mermaid extraites (pour édition sans toucher au master).

Note. Les scripts ne sont pas exécutés automatiquement par opencode. Ils sont fournis pour que l'équipe BICEC puisse régénérer les artefacts après une édition du document maître. Voir [docs/c4-architecture/render.ps1](#) et [docs/c4-architecture/render.sh](#) pour le détail.

Annexe C — Note d'autorité documentaire

Source of truth = code + ce document maître.

Ce document maître (BICEC-VERIPASS-VUE-ENSEMBLE.md) et les fichiers code/* font foi pour l'architecture, le périmètre, et le comportement courant du système. Les autres documents de docs/ restent **référence de détail** (deep-dive) lorsqu'ils sont à jour.

Les documents suivants sont **historiques** et ne doivent pas être utilisés pour définir le comportement, le périmètre, ou la roadmap :

- Tous les fichiers sous docs/diagrams/_obsolete/
- docs/agents_output/ (consolidations de prototypes)
- docs/integration-analysis/, docs/integration-analysis-gatekeeper.md
- docs/consultant-integration-prompt.md, docs/stitch-prompts.md
- Tous les diagrammes de docs/diagrams/ qui mentionnent DGI, Sopra Amplitude, ou core banking (leurs versions re-authored se trouvent dans docs/diagrams/ racine).

En cas de conflit entre ce document et un document plus ancien, **ce document prévaut** sauf mention explicite dans docs/documentation-authority.md.

Fin du document.